

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
"КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО"
НАВЧАЛЬНО-НАУКОВИХ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ
КАФЕДРА ЦИФРОВИХ ТЕХНОЛОГІЙ В ЕНЕРГЕТИЦІ

РОЗРАХУНКОВА ГРАФІЧНА РОБОТА

з дисципліни "Візуалізація графічної та геометричної інформації"

Виконав: студент групи ТР-52мп
Томін Владислав Вікторович

Перевірив: Демчишин А. А.

1. ОПИС ЗАВДАННЯ

Метою лабораторної роботи Virich Cyclic Surface є розробка WebGL-застосунку для візуалізації параметричної циклічної поверхні Віріха з використанням трикутної сітки та освітлення за моделлю Гуро. Робота спрямована на закріплення практичних навичок програмування засобами WebGL, а також на вивчення принципів побудови та відображення параметричних поверхонь у тривимірному просторі.

Поверхня Віріха є прикладом складної параметричної поверхні, що задається аналітичними математичними залежностями та має циклічний характер за параметрами u та v . У процесі виконання роботи користувач має можливість змінювати кількість кроків розбиття поверхні за параметрами $U\ Steps$ і $V\ Steps$, що дозволяє дослідити вплив щільності трикутної сітки на якість візуалізації та плавність поверхні.

Важливою частиною завдання є реалізація освітлення з використанням моделі Гуро. Дана модель передбачає обчислення освітлення у вершинах трикутної сітки з подальшою інтерполяцією кольору всередині полігонів. У роботі необхідно реалізувати фонову, дифузну та дзеркальну складові освітлення, а також застосувати дифузну, дзеркальну та normal-map текстури для покращення візуального сприйняття поверхні.

Лабораторна робота також передбачає реалізацію інтерактивного керування сценою. Користувач має змогу змінювати орієнтацію об'єкта та положення камери за допомогою миші, використовуючи механізм trackball-обертання, що дозволяє переглядати поверхню з різних ракурсів.

Окрему увагу приділено перетворенням текстурних координат. Завданням є реалізація масштабування та обертання текстури навколо довільної точки, заданої користувачем у просторі параметрів (u, v) . Керування положенням точки обертання, масштабом та кутом обертання здійснюється з клавіатури в режимі реального часу, що дає змогу спостерігати зміну вигляду текстури без перерахунку геометрії поверхні.

Для зручності користувача на сторінці передбачено інформаційну панель, яка відображає поточні значення параметрів текстурних перетворень і містить підказки щодо керування.

Результатом виконання лабораторної роботи є веб-сторінка з інтерактивною тривимірною сценою, яка дозволяє змінювати параметри геометрії та текстурних перетворень і спостерігати їх вплив на вигляд параметричної поверхні.

2. ТЕОРЕТИЧНІ ВІДОМОСТІ

2.1 Параметричні поверхні

Параметричні поверхні є одним з основних способів задання тривимірних об'єктів у комп'ютерній графіці. На відміну від полігональних моделей, які описуються лише набором вершин і граней, параметричні поверхні визначаються аналітичними залежностями між параметрами та просторовими координатами. У загальному випадку параметрична поверхня задається векторною функцією виду

$$r(u, v) = (x(u, v), y(u, v), z(u, v)),$$

де u та v — незалежні параметри, що змінюються в певних межах.

Такий підхід дозволяє компактно описувати складні геометричні форми та легко керувати їх властивостями. Зміна діапазонів або кроків параметрів u та v безпосередньо впливає на деталізацію поверхні, що є важливим для дослідження впливу роздільної здатності на якість візуалізації. Параметричні поверхні широко застосовуються у науковій візуалізації, CAD-системах та комп'ютерній графіці реального часу.

Циклічна поверхня Віріха є прикладом параметричної поверхні, що має періодичність за обома параметрами. Її форма визначається тригонометричними функціями та рядом коефіцієнтів, які задають геометричні особливості поверхні. Завдяки своїй симетрії та циклічності така поверхня є зручною для демонстрації параметричного моделювання та текстуровання.

2.2 Трикутна сітка та нормалі

Для відображення параметричних поверхонь у WebGL необхідно виконати їх апроксимацію полігональною сіткою. Найчастіше для цього використовується трикутна сітка, оскільки трикутники є базовими примітивами графічного конвеєра. Параметрична область (u, v) дискретизується на рівномірну сітку, а кожна комірка цієї сітки розбивається на два трикутники.

Кількість розбиттів за параметрами u та v визначає щільність сітки та кількість вершин. При малій кількості кроків поверхня виглядає ламаною, тоді як збільшення кількості кроків підвищує плавність, але водночас збільшує обчислювальне навантаження. Таким чином, вибір параметрів дискретизації є компромісом між якістю візуалізації та продуктивністю.

Для коректного освітлення поверхні необхідно обчислити нормалі у кожній вершині. У випадку параметричних поверхонь нормалі можуть бути визначені як векторний добуток частинних похідних за параметрами u та v . На практиці часто застосовується чисельне диференціювання, при якому похідні апроксимуються різницею координат у близьких точках параметричної області. Отримані нормалі нормалізуються та використовуються в шейдерах для розрахунку освітлення.

Окрім нормалей, для підтримки карт нормалей необхідно обчислювати дотичні вектори. Дотична зазвичай визначається вздовж одного з параметричних напрямків і разом з нормаллю та бінормаллю формує ортонормований базис, який використовується для перетворення нормалей з текстурного простору у простір сцени.

2.3 Освітлення Гуро

Модель освітлення Гуро є класичним методом зафарбовування полігональних поверхонь, який широко застосовується у графіці реального часу. Основною особливістю даної моделі є те, що освітлення обчислюється у вершинах полігональної сітки, після чого отримані значення кольору інтерполюються всередині трикутників.

У моделі Гуро зазвичай враховуються три основні складові освітлення. Фонова складова імітує непряме розсіяне світло, яке рівномірно освітлює об'єкт незалежно від його орієнтації. Дифузна складова залежить від кута між нормаллю поверхні та напрямком на джерело світла і описується законом Ламберта. Дзеркальна складова відповідає за появу відблисків і залежить від положення спостерігача відносно відбитого світлового променя.

Перевагою моделі Гуро є її обчислювальна ефективність, що робить її придатною для інтерактивних застосунків. Недоліком є можливе згладжування дрібних світлових деталей, однак у поєднанні з текстурами та картами нормалей цей недолік стає менш помітним.

2.4 Текстурні координати та їх перетворення

Текстурні координати (u, v) визначають відповідність між точками поверхні та пікселями двовимірного зображення текстури. Вони, як правило, задаються в діапазоні від 0 до 1 і інтерполюються між вершинами трикутників під час растеризації. Коректне задання текстурних координат є необхідною умовою правильного відображення текстур на поверхні.

У багатьох практичних задачах виникає потреба змінювати положення, масштаб або орієнтацію текстури без зміни геометрії об'єкта. Для цього застосовуються афінні перетворення текстурних координат, зокрема зсув, масштабування та обертання. Такі перетворення дозволяють гнучко керувати виглядом текстури без перерахунку вершин.

Обертання текстури навколо довільної точки виконується шляхом перенесення координат у нову систему відліку, виконання обертання та зворотного перенесення. Аналогічний підхід застосовується і для масштабування. Реалізація таких перетворень у фрагментному шейдері дозволяє отримати плавний результат і забезпечує ефективну роботу в реальному часі.

Інтерактивне керування текстурними перетвореннями за допомогою клавіатури дозволяє наочно спостерігати вплив параметрів масштабування та обертання на вигляд поверхні. Це робить процес вивчення текстурування більш зрозумілим і практично орієнтованим.

3. ДЕТАЛІ ВПРОВАДЖЕННЯ

Робота реалізована у вигляді веб-застосунку з використанням технології WebGL. Взаємодія з графічним контекстом здійснюється за допомогою JavaScript-коду, який відповідає за ініціалізацію сцени, створення буферів, завантаження текстур і обробку введення користувача. Відображення сцени виконується в елементі `<canvas>` HTML-сторінки.

Геометрія параметричної поверхні Віріха формується динамічно на основі значень параметрів *U Steps* та *V Steps*. Для кожної вершини обчислюються просторові координати, нормаль, дотична та текстурні координати. Побудована поверхня апроксимується трикутною сіткою, яка передається у графічний конвеєр за допомогою вершинного та індексного буферів WebGL.

У вершинному шейдері виконуються перетворення координат вершин з локального простору у простір камери та екранний простір з використанням відповідних матриць. Також у вершинному шейдері обчислюються та передаються у фрагментний шейдер нормалі, дотичні й текстурні координати. Фрагментний шейдер реалізує освітлення з урахуванням дифузної, дзеркальної та normal-map текстур, а також формує підсумковий колір фрагмента.

Масштабування та обертання текстури реалізовані у фрагментному шейдері шляхом перетворення текстурних координат відносно заданої точки (u , v). Значення координат точки *pivot* та кута обертання передаються у шейдер через *uniform*-змінні та змінюються в режимі реального часу за допомогою клавіатури. Для керування орієнтацією об'єкта використовується механізм *trackball*-обертання, що забезпечує зручну взаємодію зі сценою.

Додатково реалізовано інформаційну панель користувача, яка відображає поточні значення параметрів текстурних перетворень, зокрема координати точки *pivot*, масштаб і кут обертання. Це дозволяє контролювати стан сцени під час роботи програми та наочно спостерігати вплив змін параметрів на вигляд поверхні без необхідності звернення до вихідного коду.

4. ІНСТРУКЦІЯ КОРИСТУВАЧА

Після відкриття HTML-сторінки лабораторної роботи у веб-браузері автоматично ініціалізується WebGL-сцена та відображається параметрична циклічна поверхня Віріха. У центральній частині сторінки розташовано графічне полотно (canvas), на якому відображається тривимірна модель поверхні з накладеною текстурою та освітленням. У верхній частині сторінки розміщено панель керування параметрами геометрії, а праворуч - інформаційну панель керування текстурними перетвореннями.

Користувач може змінювати кількість кроків дискретизації поверхні за параметрами *U Steps* та *V Steps* за допомогою повзунків. Зміна цих параметрів призводить до миттєвого перерахунку геометрії поверхні. При малих значеннях кількості кроків поверхня має більш ламаний вигляд, тоді як збільшення кількості кроків забезпечує плавнішу та більш деталізовану форму. Поточні значення параметрів відображаються поруч із відповідними повзунками.

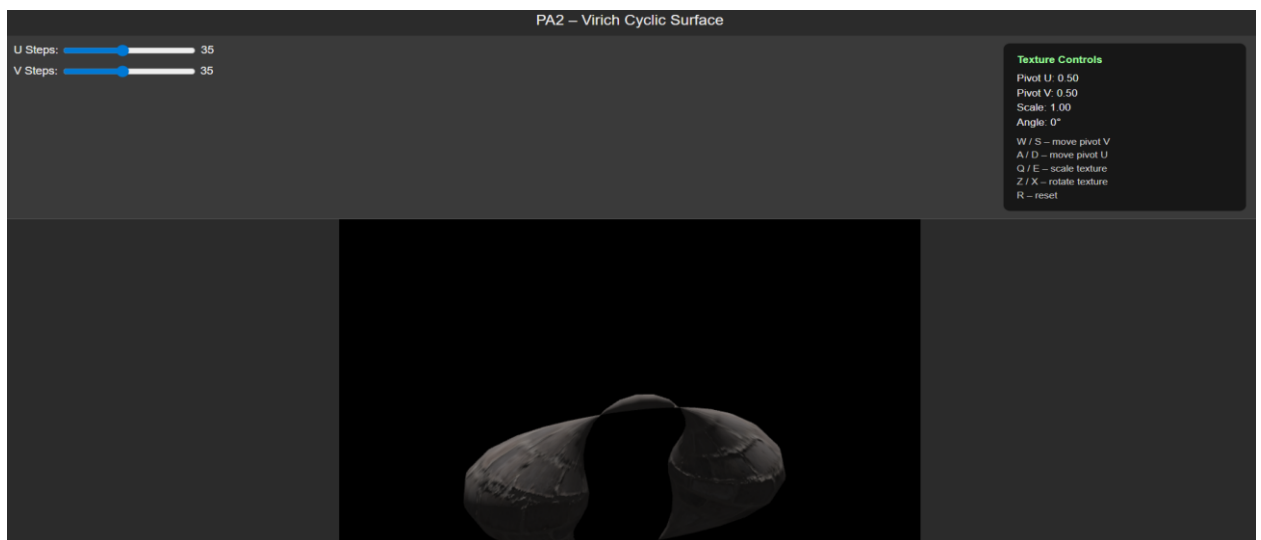


Рисунок 1 - Загальний вигляд WebGL-сцени та панелі керування параметрами поверхні

Для перегляду поверхні з різних ракурсів передбачено керування камерою за допомогою миші. При натисканні та утриманні лівої кнопки миші користувач може обертати об'єкт навколо центру сцени. Це дозволяє детально розглянути форму поверхні, оцінити розташування текстури та вплив освітлення з різних напрямків.

Окрему роль у роботі відіграє керування текстурними координатами. У правій частині сторінки розміщено інформаційну панель, у якій відображаються поточні значення координат точки *pivot* (*U*, *V*), коефіцієнта масштабування та кута обертання

текстури. Також у панелі наведено перелік клавіш керування, що спрощує взаємодію з програмою.

Переміщення точки обертання текстури здійснюється за допомогою клавіатури. Клавіші **W** та **S** відповідають за зміну положення точки pivot уздовж параметра v , тоді як клавіші **A** та **D** змінюють положення точки уздовж параметра u . Зміна координат точки відбувається в режимі реального часу, що дозволяє спостерігати, як текстура поводить себе при різному розташуванні точки pivot на поверхні.

Масштабування текстури виконується за допомогою клавіш **Q** та **E**. При натисканні клавіші **Q** відбувається зменшення масштабу текстури, а при натисканні клавіші **E** — її збільшення. Масштабування здійснюється відносно поточної точки pivot, що дозволяє зберігати її як центр перетворення та наочно оцінювати вплив масштабу на вигляд текстури.

Обертання текстури реалізовано за допомогою клавіш **Z** та **X**. Натискання цих клавіш змінює кут обертання текстури навколо заданої точки (u, v) . При цьому геометрія поверхні залишається незмінною, а всі перетворення виконуються лише над текстурними координатами. Це дозволяє чітко побачити різницю між геометричними та текстурними перетвореннями.

У будь-який момент користувач може повернути всі параметри текстурних перетворень до початкового стану, скориставшись клавішею **R**. Після цього точка pivot встановлюється у стандартне положення, масштаб текстури дорівнює одиниці, а кут обертання скидається до нульового значення. Це дає змогу швидко відновити початковий вигляд сцени та продовжити роботу.

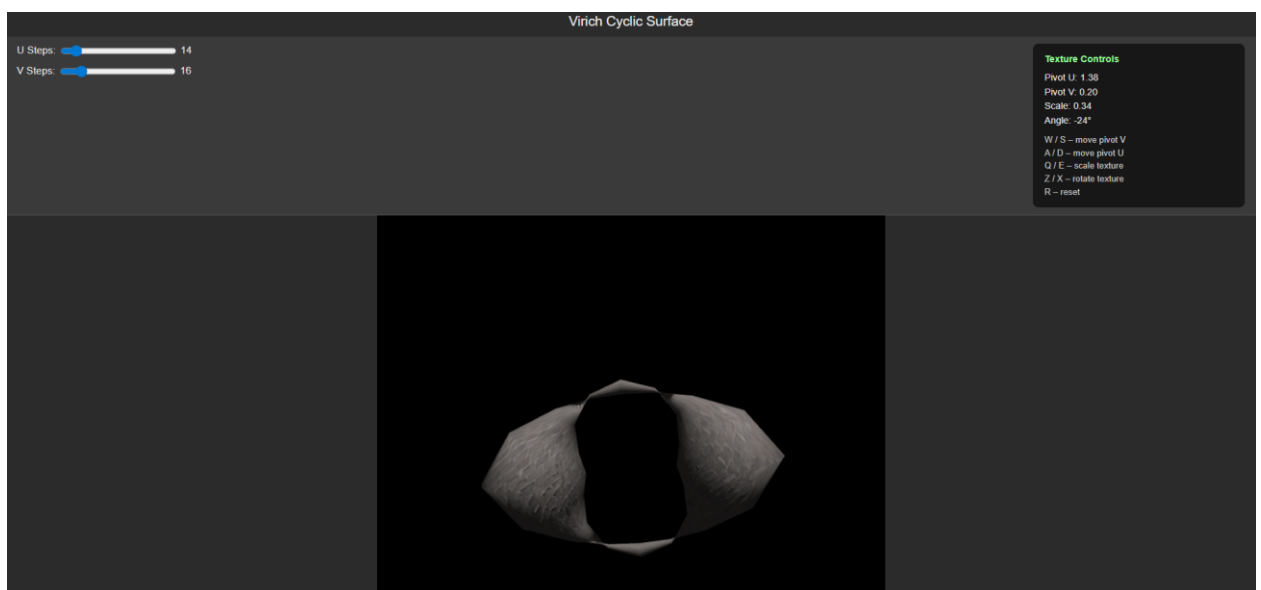


Рисунок 2 Масштабування та обертання текстури навколо довільної точки

Усі зміни параметрів виконуються інтерактивно та не потребують перезавантаження сторінки. Завдяки цьому користувач може експериментувати з різними налаштуваннями поверхні та текстури, спостерігаючи результат безпосередньо на екрані. Реалізований інтерфейс дозволяє наочно продемонструвати принципи параметричного моделювання, текстурування та перетворення текстурних координат у середовищі WebGL.

5. ВИХІДНИЙ КОД

Ініціалізація WebGL та шейдерної програми

```
const canvas = document.getElementById("webglcanvas");

gl = canvas.getContext("webgl");

program = createProgram(gl, vertexShaderSource, fragmentShaderSource);
gl.useProgram(program);

program.aVertex = gl.getAttribLocation(program, "vertex");
program.aNormal = gl.getAttribLocation(program, "normal");
program.aTangent = gl.getAttribLocation(program, "tangent");
program.aTexCoord = gl.getAttribLocation(program, "texCoord");

program.uMVP = gl.getUniformLocation(program, "ModelViewProjectionMatrix");
program.uModelView = gl.getUniformLocation(program, "ModelViewMatrix");
program.uLightPos = gl.getUniformLocation(program, "lightPos");
program.uPivot = gl.getUniformLocation(program, "uPivot");
program.uScale = gl.getUniformLocation(program, "uScale");
program.uAngle = gl.getUniformLocation(program, "uAngle");
```

Генерація та оновлення параметричної поверхні

```
function rebuildSurface() {
    let uRaw = parseInt(uSlider.value);
    let vRaw = parseInt(vSlider.value);

    let U = Math.round(10 + (uRaw * uRaw) / 100);
```

```

let V = Math.round(10 + (vRaw * vRaw) / 100);

let data = {};
CreateVirichSurfaceData(U, V, data);

if (!surface) {
    surface = new Model("VirichSurface");
}

surface.BufferData(data.verticesF32, data.indicesU16);
}

```

Обробка керування з клавіатури

```

function handleKeyDown(e) {
    const step = 0.02;

    switch(e.key.toLowerCase()) {
        case 'w': texPivotV += step; break;
        case 's': texPivotV -= step; break;
        case 'a': texPivotU -= step; break;
        case 'd': texPivotU += step; break;
        case 'q': texScale -= step; break;
        case 'e': texScale += step; break;
        case 'z': texAngle -= step * 3; break;
        case 'x': texAngle += step * 3; break;
        case 'r':
            texPivotU = 0.5;
            texPivotV = 0.5;
            texScale = 1.0;
            texAngle = 0.0;
            break;
    }
}

```

Реалізація трансформації текстурних координат у вершинному шейдері

```
vec2 uv = texCoord - uPivot;
```

```
uv = uv / uScale;
```

```
float s = sin(uAngle);
```

```
float c = cos(uAngle);
```

```
mat2 rotationMatrix = mat2(c, -s, s, c);
```

```
uv = rotationMatrix * uv;
```

```
vTexCoord = uv + uPivot;
```